

# Password Strength Analyzer

Viva / Invigilator Preparation - Question Bank with Model Answers

## 1. PROJECT OVERVIEW

### Q. Explain your project in 2 minutes.

It is a cybersecurity web application that audits passwords. It scores password strength on a 0-10 scale, estimates entropy and crack time, detects common and breached passwords, gives personalized security recommendations, stores a masked password history, generates printable reports, and provides an admin dashboard with user management and an audit log. Frontend is HTML/CSS/JS; backends are Django and PHP; database is MySQL.

### Q. Why do you have TWO backends (Django and PHP)?

Both expose the same REST-style API contract. PHP with PDO is a lightweight alternative and Django is a full framework. The frontend JavaScript automatically detects which backend is reachable and falls back gracefully, so the same UI works under both XAMPP and Django.

## 2. STRENGTH ANALYSIS LOGIC

### Q. How do you calculate password strength?

Two components: (1) Shannon entropy =  $\text{length} \times \log_2(\text{character pool size})$ , and (2) an 8-point checklist - length  $\geq 8$ , length  $\geq 12$ , lowercase, uppercase, digits, symbols, no repeated characters, and no sequential patterns. The score 0-10 maps to labels: WEAK, FAIR, GOOD, STRONG, EXCELLENT.

### Q. What is "estimated crack time"?

The number of guesses needed divided by a hypothetical 10 billion guesses per second, presented as a human label such as "180 centuries". When the search space is too large it displays "Safe for decades".

### Q. Which passwords does registration reject?

Passwords under 8 characters, passwords using fewer than 3 character classes (lowercase, uppercase, digits, symbols), passwords with triple repeats such as "aaa", passwords with sequential patterns such as "123" or "abc", and any password found in the common passwords list.

## 3. SECURITY - IMPLEMENTATION

### Q. Is the actual password ever stored?

No. Only a masked placeholder such as "Tr\*\*\*\*\*3" is stored in the history table. For login verification only the bcrypt (PBKDF2) hash is stored.

### Q. Why bcrypt and not MD5 or plain SHA-256 for passwords?

bcrypt is deliberately slow and automatically salted, which resists brute-force and rainbow-table attacks. MD5 is fast and cryptographically broken. Fast hashing functions are unsuitable for password storage.

### Q. How do you detect common passwords without revealing the plaintext?

The common password table stores only blind SHA-256 digests of the top common passwords. The server hashes the entered password and compares digests, so the database never contains the plaintext of user inputs. Breach lookup also uses HaveIBeenPwned k-anonymity: only the first 5 hex characters of the SHA-1 hash are sent, and the suffix list is compared locally.

### Q. How do you prevent SQL injection?

The PHP backend uses PDO prepared statements with `?` placeholders, and the Django backend uses the ORM. No raw user input is concatenated into SQL.

### Q. How do you prevent brute-force login attacks?

Every attempt is recorded in the login\_attempts table with IP address and user agent. After 5 failed attempts within 15 minutes the account is locked for 15 minutes. Admin bans also prevent login via the is\_active flag.

**Q. How is the admin area protected?**

The is\_admin flag is checked server-side on both backends for every admin API call, and every administrative action is written to the admin\_activity\_log audit table.

**Q. How are forms validated?**

Validation happens on both the client side (JavaScript, for instant feedback) and the server side (Django/PHP), because client checks can be bypassed.

## 4. DATABASE DESIGN

**Q. How many tables and what are the relationships?**

Seven tables: users, analyzed\_passwords, common\_passwords, recommendations, reports, login\_attempts and admin\_activity\_log. users is the parent table with one-to-many relationships to analyses, reports and logins; each recommendation optionally links back to an analyzed password.

**Q. Why MySQL with phpMyAdmin?**

MySQL was specified for the project and phpMyAdmin provides a GUI to inspect the schema, browse rows and export data, which also helps during the demo.

## 5. PRACTICAL / LIVE DEMO

**Q. Walk me through the user flow.**

Register -> login -> type a password on the analyzer -> live meter shows score, entropy, crack time and checklist -> save to history (masked only) -> get recommendations -> generate a report with export options (print, CSV, JSON) -> the admin dashboard shows totals, weekly charts and the audit log.

**Q. How would you deploy this in production?**

Run Django behind a production WSGI server such as Gunicorn with nginx or Apache, MySQL as a Windows service, enable HTTPS/TLS, disable DEBUG mode, remove demo accounts/credentials, harden session handling and CSRF, and use environment variables for secrets.

**Q. What is your future scope?**

Add OTP/2FA, breach-alert email notifications, a password generator, browser extension, password-manager import and machine-learning based strength prediction.

## 6. CORE CONCEPT DRILL

**Q. Difference between encoding, hashing and encryption?**

Encoding is reversible without a key and is used to represent data (Base64, UTF-8). Hashing is one-way and fixed length - used to verify integrity and store passwords; it cannot be reversed. Encryption is reversible but requires a secret key, and is used for confidentiality.

**Q. MD5 vs SHA-256 vs bcrypt?**

MD5 (128-bit) is broken because collisions are known. SHA-256 (256-bit) is suitable for integrity checks but is too fast for passwords. bcrypt is slow and salted, making it the correct choice for password storage.

**Q. What is a salt and why use it?**

A salt is a random value added before hashing, so identical passwords produce different hashes. It defeats rainbow tables and duplicate-hash detection.

**Q. Sessions vs cookies vs tokens?**

Cookies are small values stored by the browser. Sessions are server-side state identified by a session id held in a cookie. Tokens are self-contained credentials sent in headers - this project issues a token with a

4-hour expiry after login.

**Q. HTTP vs HTTPS?**

HTTPS is HTTP over TLS. It encrypts traffic so passwords and data are not readable in transit (prevents man-in-the-middle attacks).

**Q. Symmetric vs asymmetric encryption?**

Symmetric encryption uses one shared key (e.g., AES) and is fast. Asymmetric encryption uses a public/private key pair (e.g., RSA) and is slower - used for key exchange and digital signatures.

**Q. XSS vs SQL injection?**

XSS is a client-side attack where an attacker injects a script that runs in another user's browser; it is prevented by output escaping. SQL injection is a server-side attack where SQL is injected through user input; it is prevented by input sanitization and parameterized queries.

**Q. POST vs GET?**

GET places parameters in the URL - used for fetching, and stays in browser history. POST places parameters in the request body - used for submitting data. Passwords must never be sent via GET.

**Q. How would you add two-factor authentication?**

Implement TOTP: generate a per-user secret, show it as a QR code for Google Authenticator, and verify a time-based one-time code on login.

**Q. What happens if two users choose the same password?**

Because each account uses unique salts, the stored bcrypt hashes are completely different, so identical passwords cannot be detected from the database alone.

## 7. 30-SECOND DEMO SCRIPT

**Q. Register**

Create an account and intentionally try the bad password "123456" - show how it gets rejected as too weak/common, then register with a strong password.

**Q. Analyze**

Type "Tr0ub4dor&3" - it jumps to 7/10 with roughly 72 bits of entropy and an estimated crack time far beyond centuries. The meter, checklist and analytics react instantly.

**Q. Common check**

Type "password" - the top-common list and breach lookup flag it red, proving the system never suggests or accepts such weak credentials.

**Q. History**

Save the analysis, then open History - only a masked placeholder is stored, demonstrating that the real password never touches the database.

**Q. Report**

Generate the executive report with charts and export options (print, CSV, JSON).

**Q. Admin**

Log in as admin - user list with ban/delete, weekly activity chart and the live audit log of every recorded action.

Default logins: admin / Admin@123 | demo / Demo@123 | phpMyAdmin: root / (empty)